SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Javascript

DOM

Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

History of JavaScript

- 웹 브라우저와 JavaScript
- ECMAScript

변수

- 변수 작성 규칙
- 변수 선언 키워드

DOM

- 문서 구조
- Document 객체
- DOM Tree

DOM 선택

- 선택 메서드

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

DOM 조작

- 속성 조작
- HTML 콘텐츠 조작
- DOM 요소 조작
- Style 조작

참고

- DOM 속성 확인 Tip
- 용어 정리
- 세미콜론
- var
- 호이스팅

학습 시작

웹사이트에서 다크 모드 버튼을 누르면 화면이 어두워지는 건 어떻게 일어나는 걸까요?

1. 우리가 보는 웹 페이지는 HTML이라는 '설계도'입니다. 그리고 이 설계도만으로는 아무런 변화도 일어나지 않는, 그저 정적인 문서일 뿐입니다.
2. 이 설계도에 생명을 불어넣는 것이 바로 자바스크립트(JavaScript)입니다. 자바스크립트는 웹 페이지의 특정 요소를 찾아내고, 내용을 바꾸거나, 색깔을 입히거나, 심지어 새로운 요소를 만들어 붙이는 등 모든 동적인 조작을 할 수 있습니다.
3. 이때 자바스크립트가 HTML 설계도를 이해하고 조작할 수 있도록, 브라우저는 설계도를 'DOM(Document Object Model)'이라는 객체 트리 구조로 변환하여 제공합니다.

- 정적인 페이지에 머무르지 않고, 사용자의 행동에 따라 실시간으로 변화하는 동적인 웹 페이지를 직접 만들어 봅시다.

Objectives

학습 목표

- ✓ DOM이 무엇인지 설명하고, HTML과의 관계를 이해한다.
- ✓ let과 const를 사용해 변수와 상수를 선언할 수 있다.
- ✓ querySelector와 querySelectorAll로 DOM 요소를 선택한다.
- ✓ textContent와 setAttribute로 요소 내용과 속성을 바꾼다.
- ✓ classList 속성으로 요소의 클래스를 동적으로 조작한다.
- ✓ createElement와 appendChild로 새 DOM 요소를 추가한다.
- ✓ style 프로퍼티를 사용해 요소의 스타일을 직접 변경한다.

History of JavaScript

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

웹 브라우저와 JavaScript

웹의 탄생(1990)

- Tim Berners-Lee 경이 WWW, 하이퍼텍스트 시스템 고안하여 개발
- URL, HTTP 최초 설계 및 구현
- 초기의 웹은 정적인 텍스트 페이지만을 지원

<그림1_Javascript_DOM_웹의 탄생>

URL :
특정 웹페이지나 파일을 찾기 위한 고유한 경로 주소

<개념1_Javascript_DOM_URL>

HTTP :
웹 브라우저와 서버가 데이터를 주고 받기 위한 통신 규약

<개념2_Javascript_DOM_HTTP>

www :
World Wide Web,
전 세계적인 정보 공유 시스템

<개념3_Javascript_DOM_WWW>

웹 브라우저의 대중화(1994)

- Netscape사의 최초 상용 웹 브라우저인 Netscape Navigator 출시
- 당시 약 90% 이상의 시장 점유율을 가짐
- Netscape사는 웹의 동적인 기능을 만들기 위한 프로젝트를 시작

<그림2_Javascript_DOM_Netscape Navigator>

JavaScript의 탄생(1995)

- 당시 Netscape 소속 개발자 Brendan Eich는 회사의 요구사항이었던 웹의 동적 기능 개발을 넘어, 스크립트 언어 'Mocha'를 개발 함
- 이후 LiveScript로 이름을 변경 했으나 당시 가장 인기있던 ‘Java’의 명성에 기대보고자 ‘JavaScript’로 이름을 변경
- JavaScript는 Netscape Navigator 2.0에 탑재되어 웹 페이지에 동적 기능을 추가하는 데 사용됨

<그림3_Javascript_DOM_JavaScript의 탄생>

JavaScript 파편화(1996)

- 한편, Microsoft가 자체 웹 브라우저인 인터넷 익스플로러(IE) 3.0에 JavaScript와 유사한 언어인 'JScript'를 도입
- 이로 인해 여러 회사가 독자적으로 JavaScript 규격을 변경하여 자체 브라우저에 탑재하기 시작

➢ JavaScript 파편화의 시작

<그림4_Javascript_DOM_JavaScript의 파편화>

1차 브라우저 전쟁(1995 - 2001)

- Microsoft는 IE를 자사 윈도우 운영체제에 내장하여 무료로 배포
- 빌 게이츠를 필두로 한 Microsoft의 공격적인 마케팅, 자금력 그리고 압도적인 윈도우 운영체제 점유율 앞에 Netscape는 빠르게 몰락하기 시작
- 결국 IE의 시장 점유율은 2002년 약 96%에 달하며 Microsoft가 승리
- 훗날 Brendan Eich와 Netscape의 핵심 개발진은 모질라 재단을 설립(2003)하여 Firefox 브라우저를 출시함 (2004)

<그림5_Javascript_DOM_1차 브라우저 전쟁>

1차 브라우저 전쟁의 영향

- 웹 표준의 부재로 인해 각 기업에서 자체 표준을 확립하려는 상황 발생

➢ 이는 웹 개발자들에게 큰 혼란을 주었고, 결과적으로 웹 표준의 중요성을 인식하는 계기가 됨

TIP

- 웹 표준이 중요한 이유
  - 브라우저나 기기에 상관없이 웹 사이트가 동일하게 작동하여, 일관된 사용자 경험을 제공합니다.
  - 브라우저별 코드를 따로 작성할 필요가 없어 개발 시간과 비용이 줄고, 유지보수가 쉬워집니다.

ECMAScript 출시 (1997)

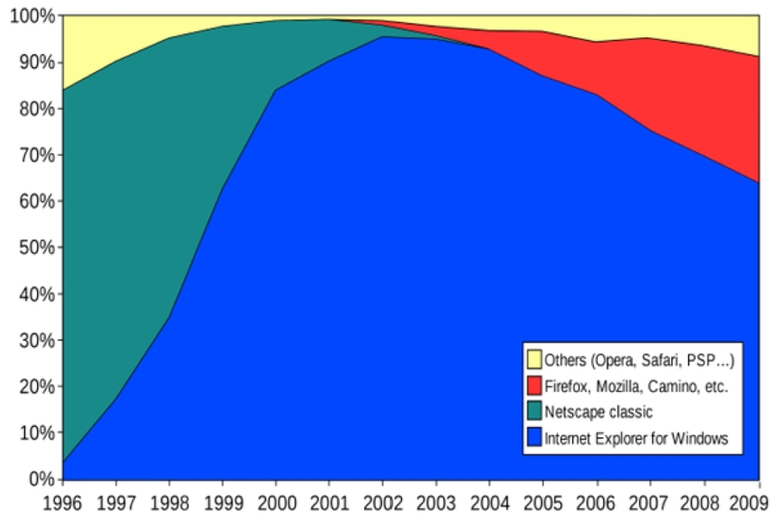
- JavaScript의 파편화를 막기 위해 Netscape사는 ECMA 재단에 웹 표준 제작을 요청
- ECMA에서 ECMAScript라는 표준 언어를 정의하여 발표 (1997)
- 이때부터 JavaScript는 ECMAScript 표준에 기반을 두고 발전하기 시작

<그림6_Javascript_DOM_ECMAScript>

2차 브라우저 전쟁(2004-2017)

- 웹 표준이 정의되었지만 당시 가장 높은 점유율을 가진 IE는 웹 표준을 지키지 않았고 독자적인 규격을 유지하며 웹 시장을 주도
- IE의 독주에 Firefox가 대항하기 시작
- 2008년까지 30% 점유율 차지

<그림7_Javascript_DOM_IE>



<그림8_Javascript_DOM_2차 브라우저 전쟁 점유율>

Chrome 브라우저의 등장 (2008)

- Google의 Chrome 브라우저 출시
- Chrome은 출시 3년 만에 Firefox의 점유율을 넘어서고 그로부터 반년 뒤 IE의 점유율을 넘어섬

<그림9_Javascript_DOM_Chrome 브라우저 등장>

Chrome이 시장 우위를 점하게 된 이유

- 빠른 성능, 다양한 플랫폼 지원, 보안, Google 생태계 통합 등 다양한 이유가 있지만,
  가장 결정적인 이유는 바로 “적극적인 웹 표준 준수”
- 호환성
  - 웹 표준을 준수함으로써, 사용자들은 브라우저 간에 일관된 웹 페이지 볼 수 있게 됨
  - 이는 다양한 플랫폼 및 기기에서 웹 사이트가 일관되게 동작할 수 있음을 의미
- 강력한 개발자 도구
  - 웹 개발자를 위한 강력한 도구를 제공하여 웹 애플리케이션을 개발하는 데 도움

2차 브라우저 전쟁의 영향

- Chrome이 웹 표준을 준수하고 새로운 웹 기술을 적극적으로 채택함으로써,
  다른 브라우저 제조사들도 웹 표준 준수에 대한 중요성을 인식하고
  이에 따라 웹 표준을 채택하는 데 더 많은 노력을 기울이게 됨
- 웹의 기능이 크게 확장되며 웹 애플리케이션의 비약적인 발전을 이끌어 감

➢ 2차 브라우저 전쟁이 남긴 “웹의 기술적 발전과 웹 표준의 중요성”

V8 자바스크립트 엔진 :
구글이 개발한 고성능 엔진으로 웹을
문서에서 어플리케이션 플랫폼으로 바꾸는 데
결정적인 역할을 했습니다.

<개념4_Javascript_DOM_V8 자바스크립트 엔진>

TIP

- Chrome은 단순히 표준만 잘 지킨 것이 아니라, 자체 개발한 V8 자바스크립트 엔진은 압도적인 성능을 보여줬습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

ECMAScript

ECMAScript

- Ecma International이 정의한 “표준화된 스크립트 프로그래밍 언어 명세”

➢ 스크립트 언어가 준수해야 하는 규칙, 세부사항 등을 정의

TIP

- 프로그래밍 언어 명세
  - 언어의 규칙 설명서로써 해당 언어의 문법과 코드가 어떻게 동작해야 하는지를 정의한 설명서입니다.
  - 기준이 되는 설계도로써 코드가 모든 환경에서 동일하게 동작하도록 보장합니다.

ECMAScript와 JavaScript

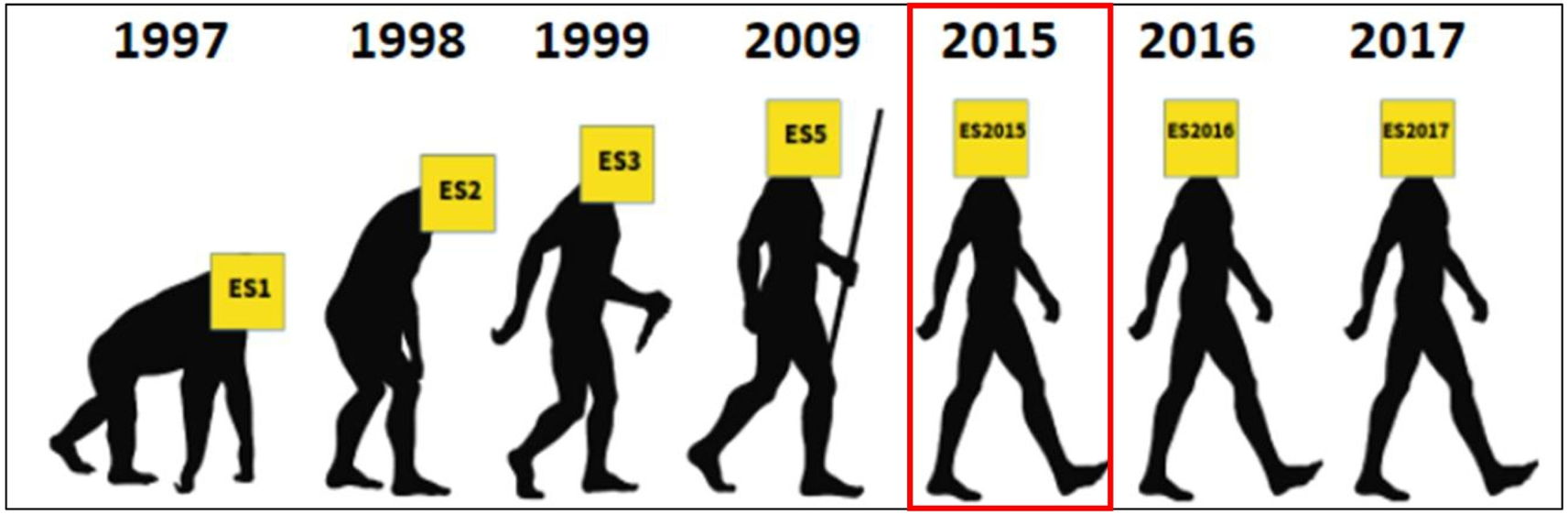
- JavaScript는 ECMAScript 표준을 구현한 구체적인 프로그래밍 언어
- ECMAScript의 명세를 기반으로 하여 웹 브라우저나 Node.js와 같은 환경에서 실행됨

➢ECMAScript는 JavaScript의 표준이며, JavaScript는 ECMAScript 표준을 따르는 구체적인 프로그래밍 언어

➢ECMAScript는 언어의 핵심을 정의하고, JavaScript는 ECMAScript 표준을 따라 구현된 언어로 사용됨

ECMAScript의 역사

- ECMAScript 5(ES5)에서 안정성과 생산성을 크게 높임(2009)
- ECMAScript 2015(ES6)에서 객체지향 프로그래밍 언어로써 많은 발전을 이루어, 역사상 가장 중요한 버전으로 평가됨(2015)



<그림10_Javascript_DOM_ECMAScript 역사>

JavaScript의 현재와 미래

- 현재는 Chrome, Firefox, Safari, Microsoft Edge 등 다양한 웹 브라우저가 경쟁하고 있으며, 모바일 등 시장이 다양화 되어있음
- 과거에는 JavaScript가 브라우저 내에서 웹 페이지의 동적인 기능을 구현하는 용도로만 사용되었음
  - 예를 들어, 사용자의 입력에 따라 웹 페이지의 내용이 동적으로 변경되거나, 애니메이션 효과가 적용되는 등의 기능
- 이후 Node.js(2009 출시)로 인해 브라우저 외부에서도 실행 가능해져 서버 사이드 개발에도 사용되기 시작함
- 다양한 프레임워크와 라이브러리들이 개발되면서, 웹 개발 분야에서는 필수적인 언어로 자리 잡게 됨

변수

JavaScript 문법 학습

- ECMAScript 2015 (ES6) 이후의 문법을 기준으로 설명
- 권장 스타일 가이드
  - https://standardjs.com/rules-kokr.html

<그림11_Javascript_DOM_JavaScript 스타일 가이드>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

변수 작성 규칙

식별자(변수명) 작성 규칙

- 반드시 문자, 달러(‘$’) 또는 밑줄(‘_’)로 시작
- 대소문자를 구분
- 예약어 사용 불가
  - `for, if, function` 등

예약어:
프로그래밍 언어가 문법을 위해 미리 예약해둔 특별한 단어들
(ex: for, if, function, …)

<개념5_Javascript_DOM_예약어>

TIP

- 의미가 드러나는 이름을 사용하여 데이터의 의미를 명확히 설명하세요.(ex: userName, phoneNumber)
- ‘$’ 나 ‘_’로 시작하는 변수는 특별한 용도(DOM 선택, 내부용 변수)로 쓰이니, 일반 변수와 구분하세요.

식별자(변수명) Naming Convention

- 카멜 케이스(camelCase)
  - 변수, 객체, 함수에 사용
- 파스칼 케이스(PascalCase)
  - 클래스, 생성자에 사용
- 대문자 스네이크 케이스(SNAKE_CASE)
  - 상수(constants)에 사용

상수(constants):
한 번 선언하면 값을 바꿀 수 없도록 잠가버리는 특별한 변수

<개념6_Javascript_DOM_상수(constants)>

TIP

- 문법이 아닌 '소통'의 규칙임으로 소통 도구임을 이해해야 합니다.
- 가장 중요한 것은 '일관성'이고, 팀의 스타일 가이드를 따르는 것이 우선입니다.

변수 선언 키워드

변수 선언 키워드 3가지

1. `let`
   - 재할당이 필요한 변수를 선언할 때 사용

2. `const`
   - 재할당이 불가능한 상수를 선언할 때 사용

~~3. var~~
   - 재선언/재할당이 가능하고, 현재는 **호이스팅(Hoisting)** 문제로 사용을 권장하지 않음

참고 챕터에서 학습 예정

let

let

블록 스코프(block scope)를 갖는
지역 변수를 선언

1. 재할당 가능
2. 재선언 불가능
3. ES6에서 추가

```
let number = 10  // 1. 선언 및 초기값 할당
number = 20      // 2. 재할당
```

```
let number = 10  // 1. 선언 및 초기값 할당
let number = 20  // 2. 재선언 불가능
```

const

const

블록 스코프(block scope)를 갖는
지역 변수를 선언

1. 재할당 불가능
2. 재선언 불가능
3. ES6에서 추가

```
const number = 10  // 1. 선언 및 초기값 할당
number = 10        // 2. 재할당 불가능
```

```
const number = 10  // 1. 선언 및 초기값 할당
const number = 20  // 2. 재선언 불가능
const number  // SyntaxError
```

블록 스코프(block scope)

- if, for, 함수 등의 '중괄호({}) 내부'를 가리킴
- 블록 스코프를 가지는 변수는 블록 바깥에서 접근 불가능

```
<!-- js_variable.html -->
let x = 1
if (x === 1) {
    let x = 2
    console.log(x)  // 2
}
console.log(x)  // 1
```

TIP

- 스코프 안에서 'let' 으로 선언하면 밖에서 접근할 수 없습니다.
- 'var' 는 블록을 무시하고 함수 단위로 작동해 파이썬과 비슷합니다. 가급적 사용하지 않는 것을 권장합니다.

어떤 변수 선언 키워드를 사용해야 할까?

- "const를 기본으로 사용할 것"
  - 코드의 의도 명확화
    - 해당 변수가 재할당되지 않을 것임을 명확히 표현
    - 개발자들에게 변수의 용도와 동작을 더 쉽게 이해할 수 있게 해줌
  - 버그 예방
    - 의도치 않은 변수 값 변경으로 인한 버그를 예방
    - 큰 규모의 프로젝트나 팀 작업에서 중요
- 필요한 경우에만 `let`으로 전환(재할당이 필요한 경우)
  - `let`을 사용하는 것은 해당 변수가 의도적으로 변경될 수 있음을 명확히 나타냄
  - 코드의 유연성을 확보하면서도 `const`의 장점을 최대한 활용할 수 있음

DOM

웹 브라우저에서의 JavaScript

- 웹 페이지에서 동적인 기능을 담당

1. HTML script 태그

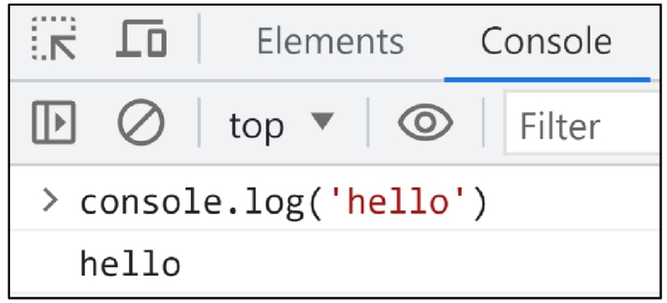
```
<body>
  <script>
    console.log('hello')
  </script>
</body>
```

2. js 확장자 파일

```
// hello.js
console.log('hello')
```

```
<body>
  <script src="hello.js"></script>
</body>
```

3. 브라우저 Console



<그림12_Javascript_DOM_브라우저 console>

SAMSUNG SW AI ACADEMY FOR YOUTH

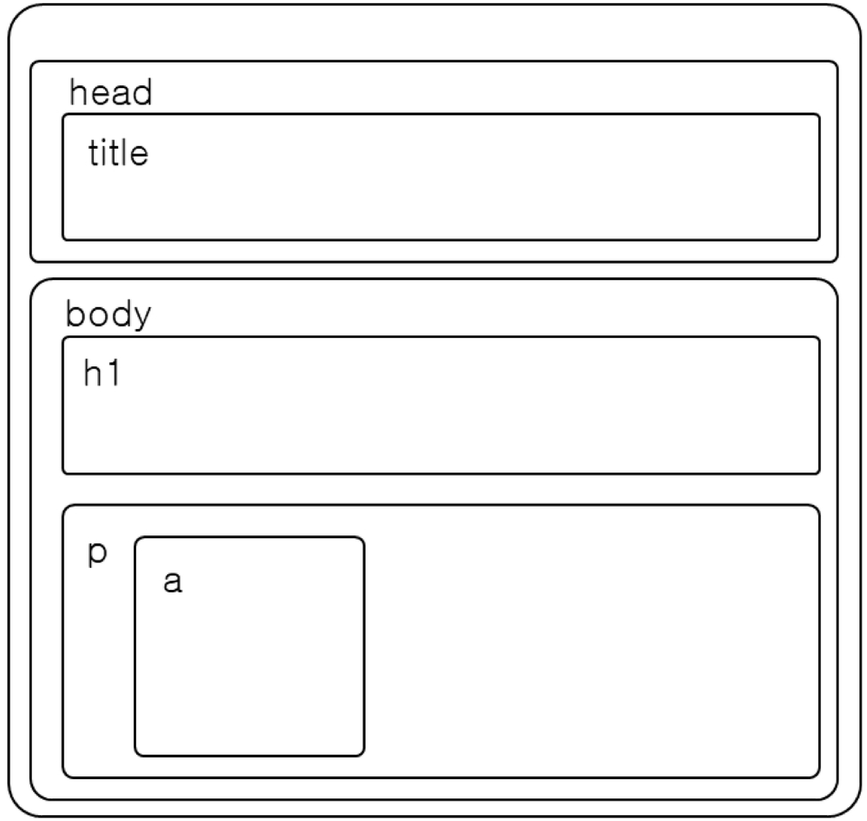
함께가요 미래로!
Enabling People

문서 구조

Document structure (1/2)

- HTML 문서는 여러 상자가 중첩된 구조로 볼 수 있음

```
<!DOCTYPE html>
<html lang="en"><head>
  <title>Document</title>
</head><body>
  <h1>Heading</h1>
  <p>Hello,
    <a href="https://www.google.com/">google</a>
  </p>
</body></html>
```



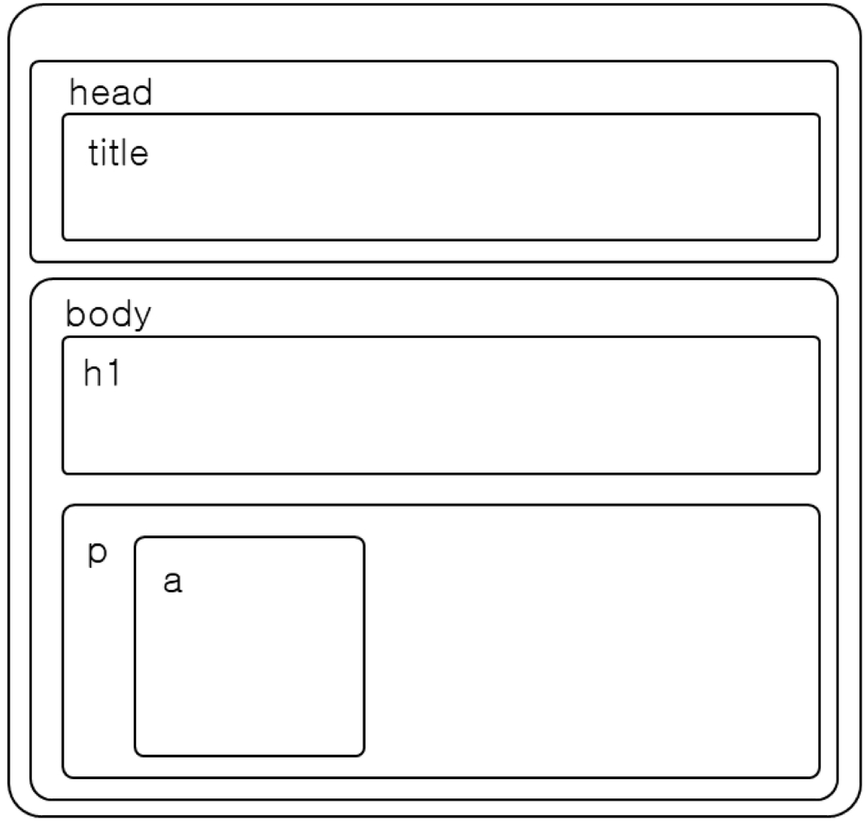
<그림13_Javascript_DOM_HTML 문서 구조>

Document structure (2/2)

- 브라우저가 문서를 표현하기 위해 사용하는 데이터 구조는
  우측 이미지와 같은 모양을 가짐

- 각 상자는 객체이며 개발자는 이 객체와 상호작용하여
  어떤 HTML 태그를 나타내는지, 어떤 콘텐츠가 포함되어 있는지
  등을 알아낼 수 있음

- 이 표현을 객체 모델(Document Object Model)
  줄여서 DOM이라고 부름



<그림13_Javascript_DOM_HTML 문서 구조>

DOM

DOM

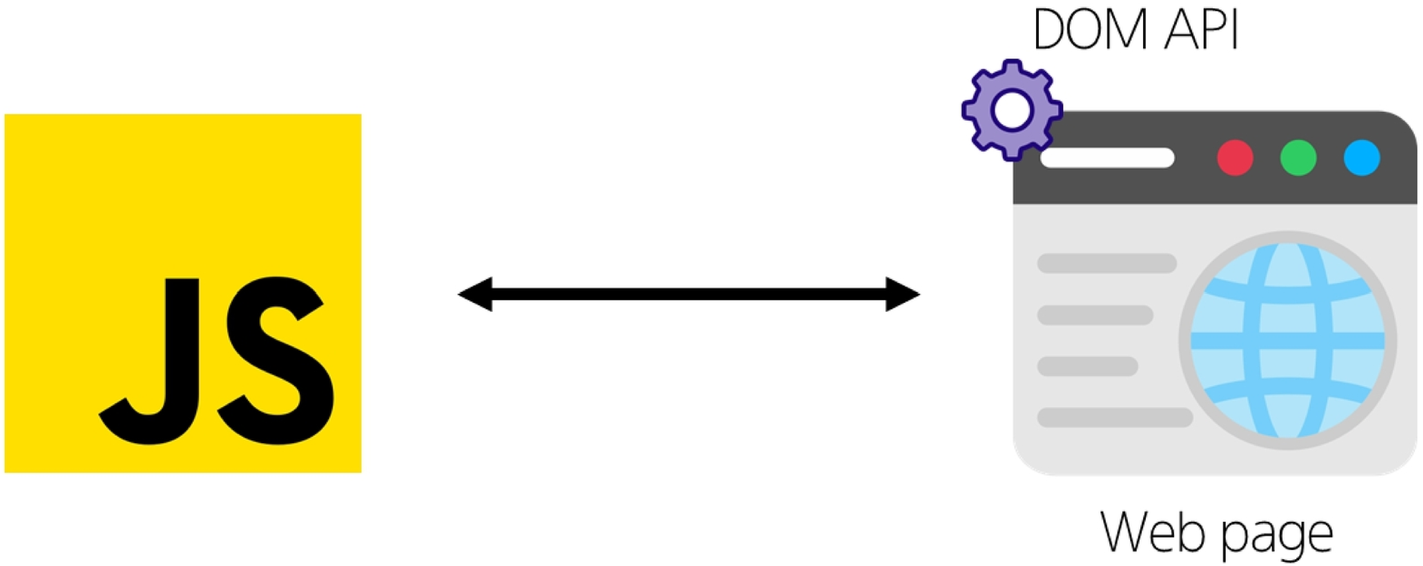
The Document Object Model

웹 페이지(Document)를 구조화된 객체로 제공하여
프로그래밍 언어가 페이지 구조에 접근할 수 있는 방법을 제공

문서 구조, 스타일, 내용 등을 변경할 수 있도록 함

DOM API

- 다른 프로그래밍 언어가 웹 페이지에 접근 및 조작 할 수 있도록, 페이지 요소들을 객체 형태로 제공하며 관련된 메서드(Method)도 함께 제공
- HTML 구조와 내용을 조작하는 명령어 모음



<그림14_Javascript_DOM_DOM API>

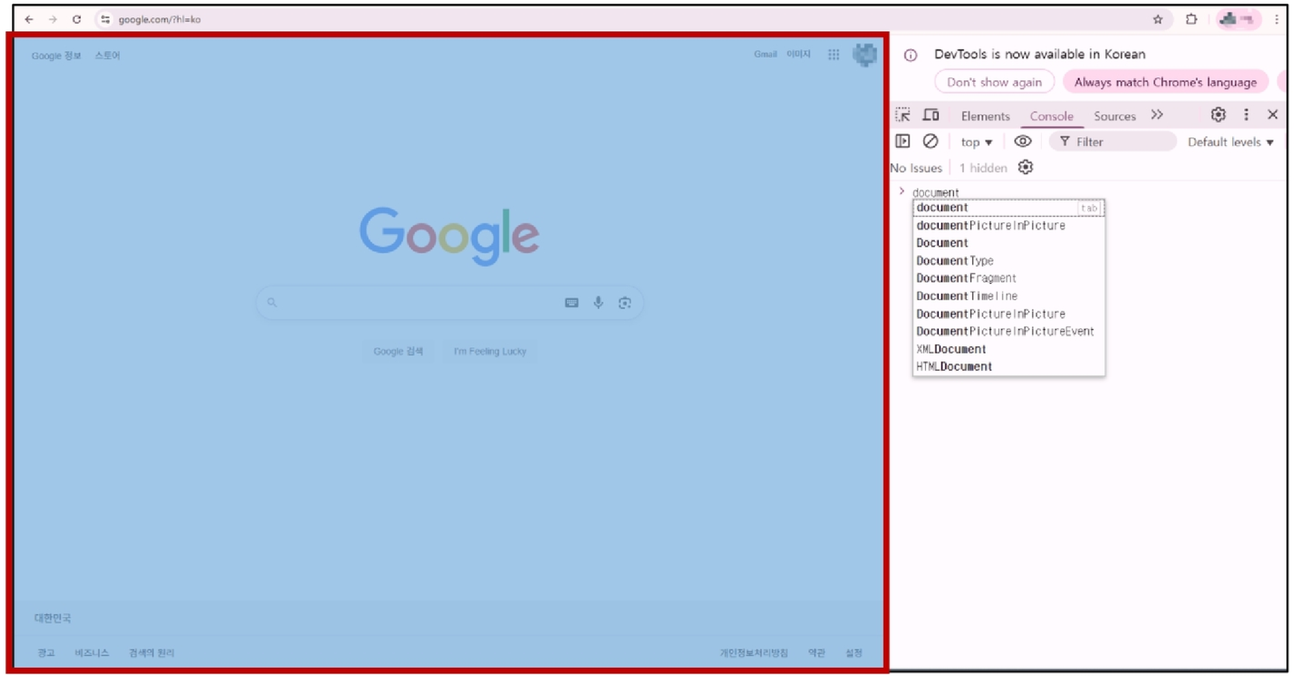
document 객체

document 객체 (1/2)

- 웹 페이지를 나타내는 DOM 트리의 최상위 객체
- HTML 문서의 모든 콘텐츠에 접근하고 조작할 수 있는 진입점

DOM 트리:
HTML 태그들의 관계를 나무 모양으로 만든 계층 구조

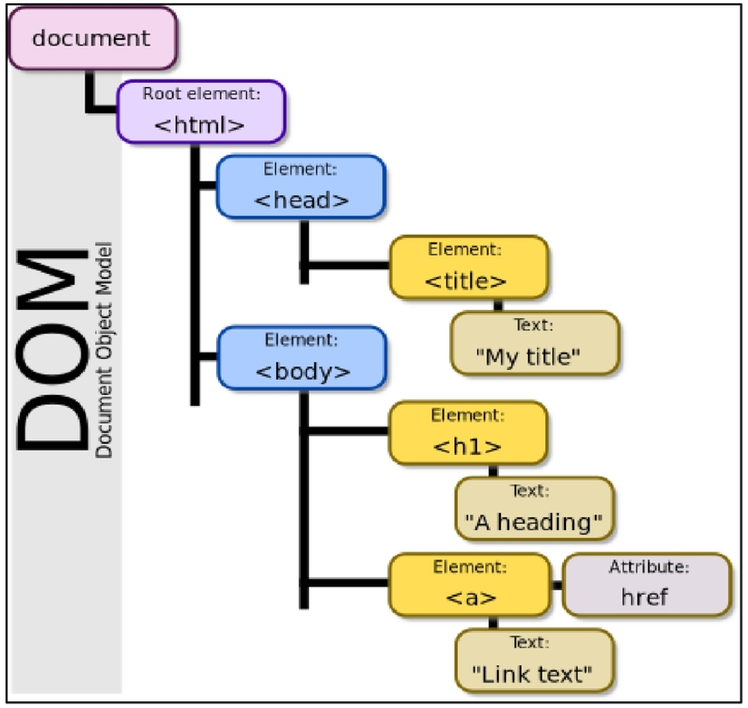
<개념7_Javascript_DOM_DOM 트리>



<그림15_Javascript_DOM_DOM 트리>

document 객체 (2/2)

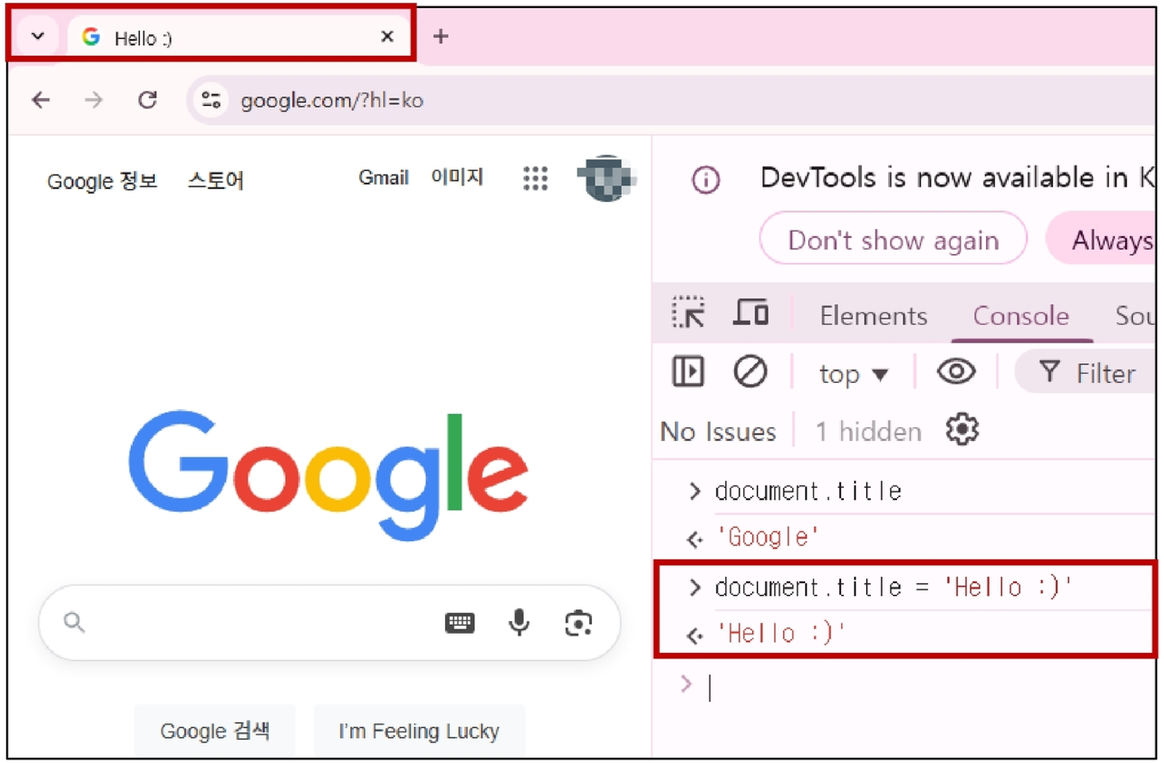
- DOM에서 모든 요소, 속성, 텍스트는 하나의 객체
- 모두 document 객체의 하위 객체로 구성됨



<그림16_Javascript_DOM_document 객체>

document 객체 예시

- HTML의 <title> 변경하기



<그림17_Javascript_DOM_HTML title 변경 예시>

SAMSUNG SW AI ACADEMY FOR YOUTH

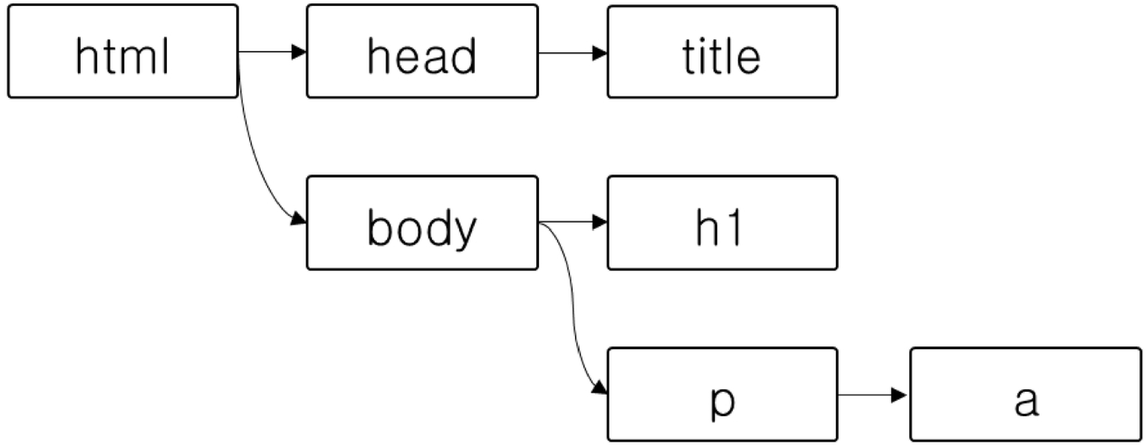
함께가요 미래로!
Enabling People

DOM Tree

DOM tree

- HTML 태그를 나타내는 elements의 node는 문서의 구조를 결정
- 이들은 다시 자식 node를 가질 수 있음 (ex. document.body)
  - ➢ 객체 간 상속 구조가 존재

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document</title>
</head>
<body>
  <h1>Heading</h1>
  <p>
    Hello,<a href="https://www.google.com/">google</a>
  </p>
</body>
</html>
```



<그림18_Javascript_DOM_Dom tree 간 상속 구조 예>

핵심 포인트

DOM 핵심

문서의 요소들을 객체로 제공하여 다른 프로그래밍 언어에서
접근하고 조작할 수 있는 방법을 제공하는 API

TIP

- DOM은 문서를 부모-자식 관계의 계층적인 트리 구조로 표현
- DOM 조작은 웹페이지에 실시간으로 반영되어, 사용자와 상호작용하는 동적 페이지를 만듭니다.
- 사용자의 클릭이나 키보드 입력같은 이벤트를 감지하고, 이에 반응하는 상호작용을 만드는 기반이 됩니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DOM 선택

DOM 조작 시 기억해야 할 것

웹 페이지를 동적으로 만들기 == **웹 페이지를 조작하기**

조작 순서

1. 조작 하고자 하는 요소를 선택 (또는 탐색)
2. 선택된 요소의 콘텐츠 또는 속성을 조작

선택 메서드

document.querySelector(selector)

- 요소 한 개 선택
- 제공한 선택자(selector)와 일치하는 첫 번째 요소를 하나 선택
- 제공한 선택자를 만족하는 첫 번째 element 객체를 반환 (없다면 null 반환)

selector:
찾고 싶은 html 요소를 지정하는 검색 조건 (ex: '.content ', '#id', …)

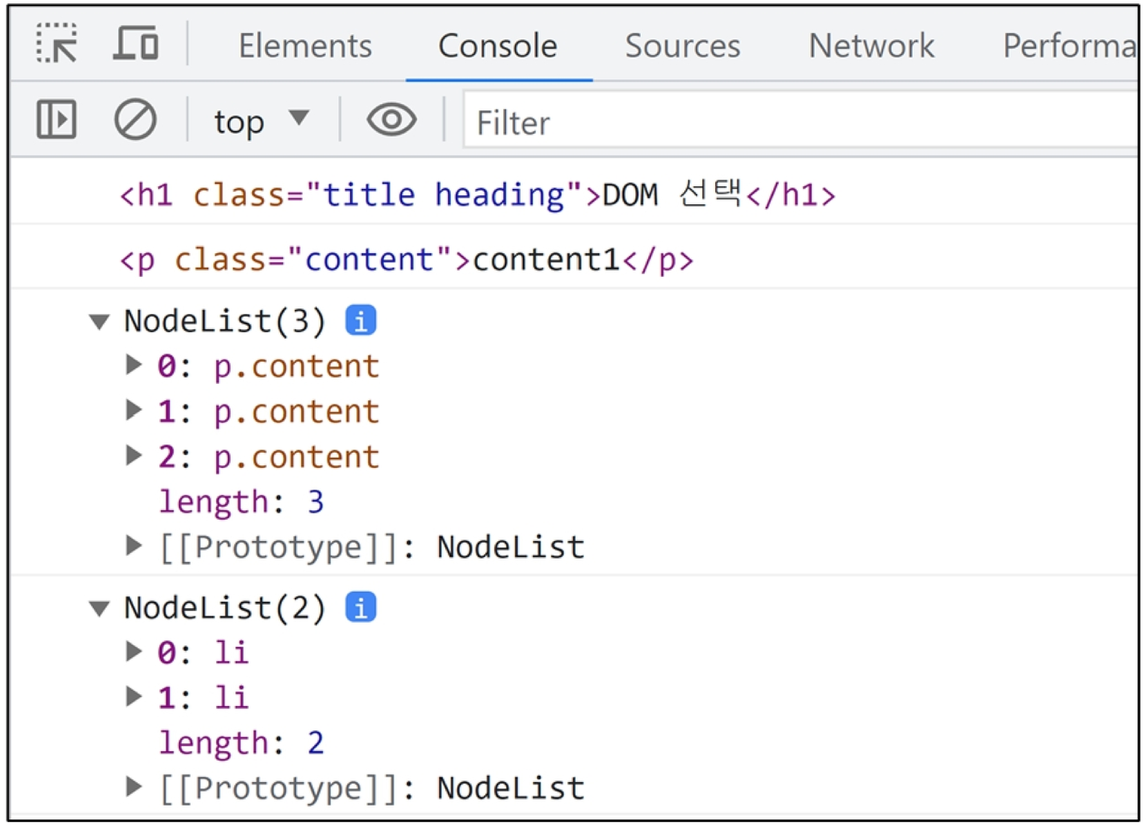
<개념8_Javascript_DOM_selector>

document.querySelectorAll(selector)

- 요소 여러 개 선택
- 제공한 선택자와 일치하는 여러 element를 선택
- 제공한 선택자를 만족하는 NodeList를 반환

DOM 선택 실습

```
<!-- select.html -->
<body>
  <h1 class="heading">DOM 선택</h1>
  <a href="https://www.google.com/">google</a>
  <p class="content">content1</p>
  <p class="content">content2</p>
  <p class="content">content3</p>
  <ul>
    <li>list1</li>
    <li>list2</li>
  </ul>
  <script>
    console.log(document.querySelector('.heading'))
    console.log(document.querySelector('.content'))
    console.log(document.querySelectorAll('.content'))
    console.log(document.querySelectorAll('ul > li'))
  </script>
</body>
```



<그림19_Javascript_DOM_DOM 선택 실습>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DOM 조작

DOM 조작

1. 속성(attribute) 조작
    - 클래스 속성 조작
    - 일반 속성 조작
2. HTML 콘텐츠 조작
3. DOM 요소 조작
4. 스타일 조작

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

속성 조작

속성 조작

1. 클래스(Class) 속성 조작
   - 스타일링 및 상태 제어를 위한 클래스 목록을 동적으로 추가/제거

2. 일반 속성(Attribute) 조작
   - id, href 등 요소의 모든 HTML 속성 값을 직접 설정/조회

클래스 속성 조작

‘classList’ property

요소의 클래스 목록을
DOMTokenList(유사 배열) 형태로 반환

- HTML 요소의 클래스 목록을 쉽게 제어(추가/제거)하는 도구
- [참고] https://developer.mozilla.org/ko/docs/Web/API/Element/classList

1. classList 메서드

- element.classList.add()
  - 지정한 클래스 값을 추가
- element.classList.remove()
  - 지정한 클래스 값을 제거
- element.classList.toggle()
  - 클래스가 존재한다면 제거하고 false를 반환 (존재하지 않으면 클래스를 추가하고 true를 반환)

toggle:
있으면 제거하고, 없으면 추가하는 식으로 상태를 바꾸는 기능

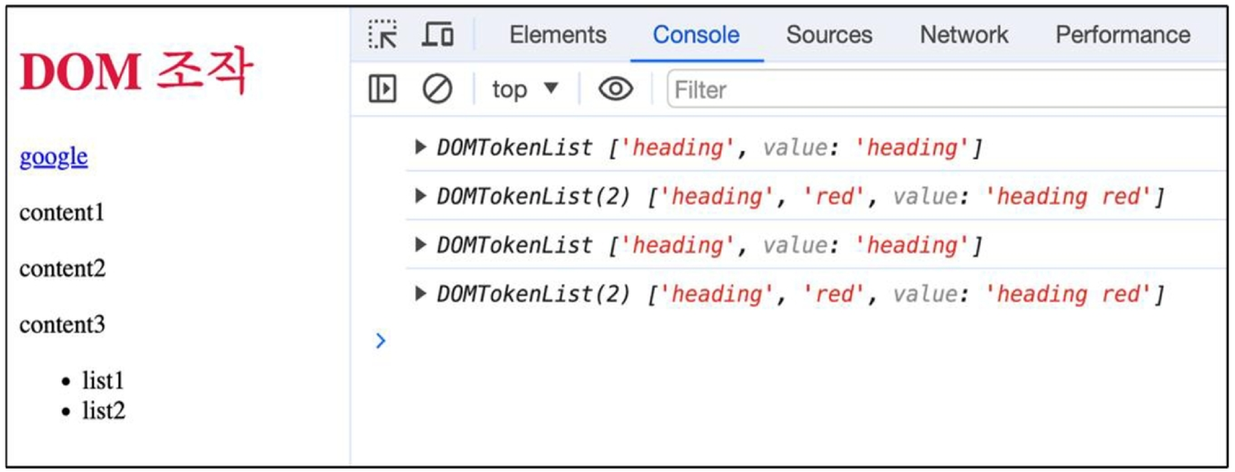
<개념9_Javascript_DOM_toggle>

클래스 속성 조작 실습 (1/2)

- add()와 remove() 메서드를 사용해 지정한 클래스 값을 추가 혹은 제거

```
<!-- element_manipulation.html -->

<style>
  .red {
    color: crimson;
  }
</style>
```



<그림20_Javascript_DOM_클래스 속성 조작 실습>

클래스 속성 조작 실습 (2/2)

- add()와 remove() 메서드를 사용해 지정한 클래스 값을 추가 혹은 제거

```
<!-- element_manipulation.html -->

const h1Tag = document.querySelector('.heading')
console.log(h1Tag.classList)

h1Tag.classList.add('red')
console.log(h1Tag.classList)

h1Tag.classList.remove('red')
console.log(h1Tag.classList)

h1Tag.classList.toggle('red')
console.log(h1Tag.classList)
```

2. 일반 속성 조작 메서드

- `Element.getAttribute()`
  - 해당 요소에 지정된 값을 반환 (조회)
- `Element.setAttribute(name, value)`
  - 지정된 요소의 속성 값을 설정
  - 속성이 이미 있으면 기존 값을 갱신 (그렇지 않으면 지정된 이름과 값으로 새 속성이 추가)
- `Element.removeAttribute()`
  - 요소에서 지정된 이름을 가진 속성 제거

TIP

- **getAttribute()는 HTML 초기값을, .value와 같은 프로퍼티는 현재 상태 값을 반환하니 주의하세요.**
- **setAttribute()는 숫자나 boolean 값을 넣어도 모두 문자열로 변환하여 저장하니 타입에 주의하세요.**

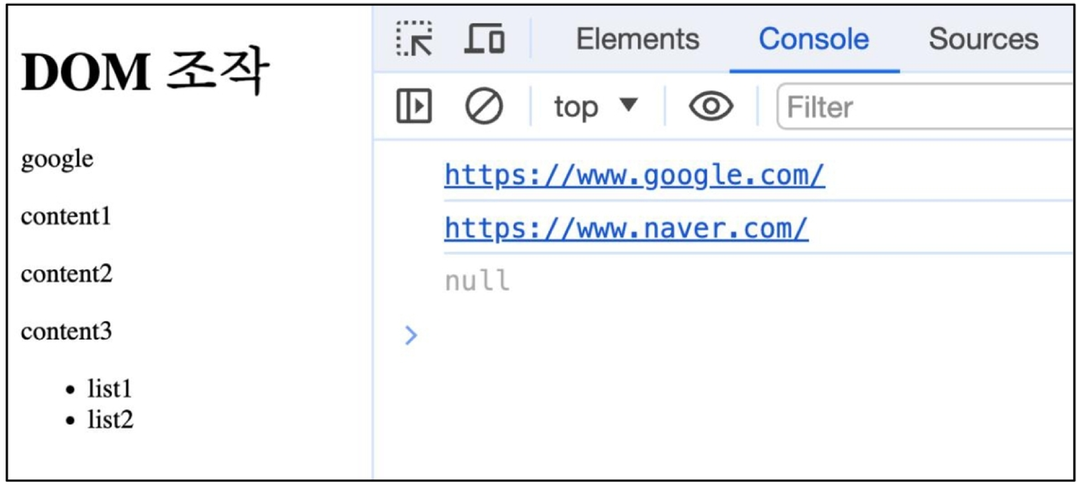
2. 일반 속성 조작 실습

```
<!-- element_manipulation.html -->

const aTag = document.querySelector('a')
console.log(aTag.getAttribute('href'))

aTag.setAttribute('href', 'https://www.naver.com/')
console.log(aTag.getAttribute('href'))

aTag.removeAttribute('href')
console.log(aTag.getAttribute('href'))
```



<그림21_Javascript_DOM_일반 속성 조작 실습>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

HTML 콘텐츠 조작

HTML 콘텐츠 조작

‘textContent’ property

요소의 텍스트 콘텐츠를 표현

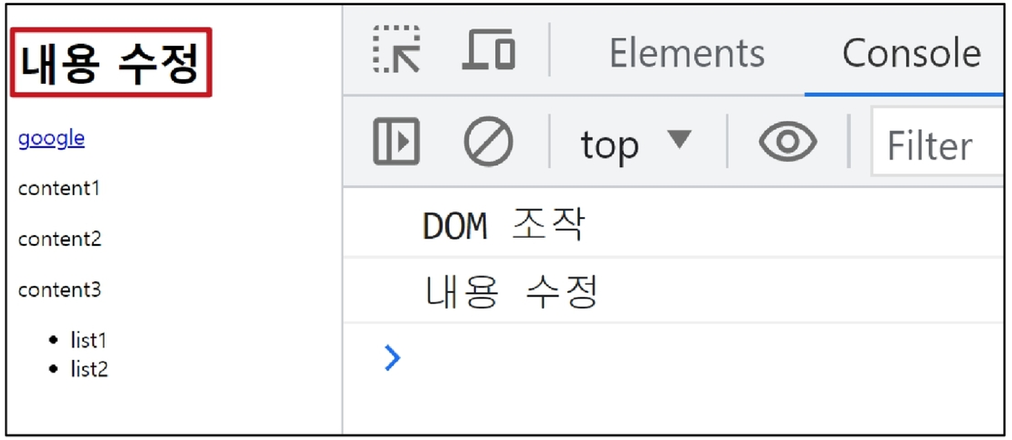
- HTML 태그를 완전히 제거하고 순수한 텍스트 데이터만 얻고 싶을 때 가장 유용합니다.
- 예) `<p>Hello <b>World</b></p>` ➔ textContent는 `'Hello World'`

HTML 콘텐츠 조작 실습

```
<!-- contents_manipulation.html -->

const h1Tag = document.querySelector('.heading')
console.log(h1Tag.textContent)

h1Tag.textContent = '내용 수정'
console.log(h1Tag.textContent)
```



<그림22_Javascript_DOM_콘텐츠 조작 실습>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DOM 요소 조작

DOM 요소 조작 메서드

- `document.createElement(tagName)`
  - 작성한 tagName의 HTML 요소를 생성하여 반환
- `Node.appendChild()`
  - 한 Node를 특정 부모 Node의 자식 NodeList 중 마지막 자식으로 삽입
  - 추가된 Node 객체를 반환
- `Node.removeChild()`
  - DOM에서 자식 Node를 제거
  - 제거된 Node를 반환

TIP

- ‘createElement’ 는 메모리에만 요소를 만들고, 실제로는 ‘appendChild’로 문서에 삽입해야 화면에 보입니다.
- ‘removeChild’로 제거된 노드는 메모리에 남아있어, 변수에 담아두면 언제든지 재사용이 가능합니다.

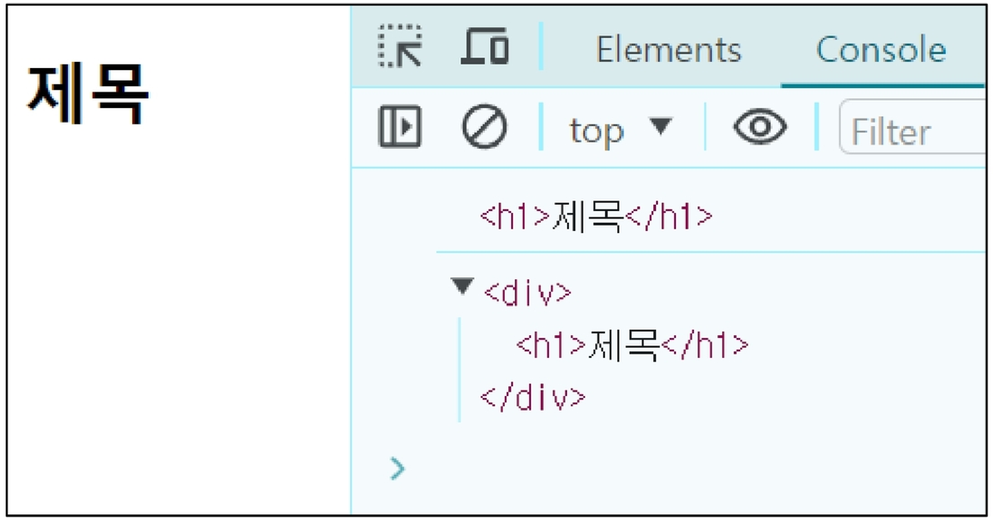
DOM 요소 조작 실습

```
<!-- dom_manipulation.html -->

// 생성
const h1Tag = document.createElement('h1')
h1Tag.textContent = '제목'
console.log(h1Tag)

// 추가
const divTag = document.querySelector('div')
divTag.appendChild(h1Tag)
console.log(divTag)

// 삭제
const pTag = document.querySelector('p')
divTag.removeChild(pTag)
```



<그림23_Javascript_DOM_DOM 요소 조작 실습>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

style 조작

style 조작

‘style’ property | 해당 요소의 모든 style 속성 목록을 포함하는 속성

TIP

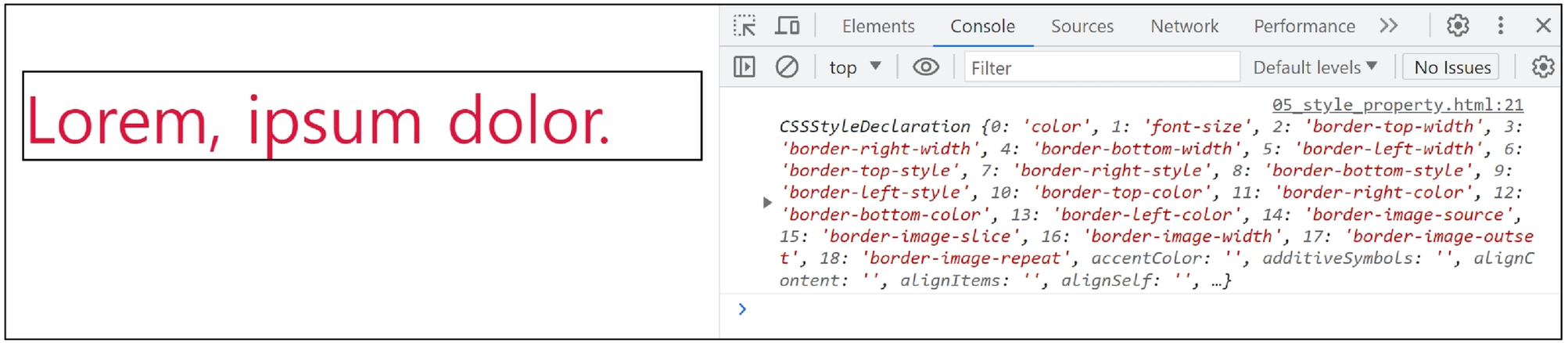
- ‘style’ property 는 HTML 태그에 style=“…” 속성을 직접 추가/수정하는 것과 같습니다.
- 여러 스타일 변경은 가급적 클래스(class) 활용하는 것을 추천합니다.

style 조작 실습

```
<!-- style_property.html -->

const pTag = document.querySelector('p’)

pTag.style.color = 'crimson'
pTag.style.fontSize = '2rem'
pTag.style.border = '1px solid black'
console.log(pTag.style)
```



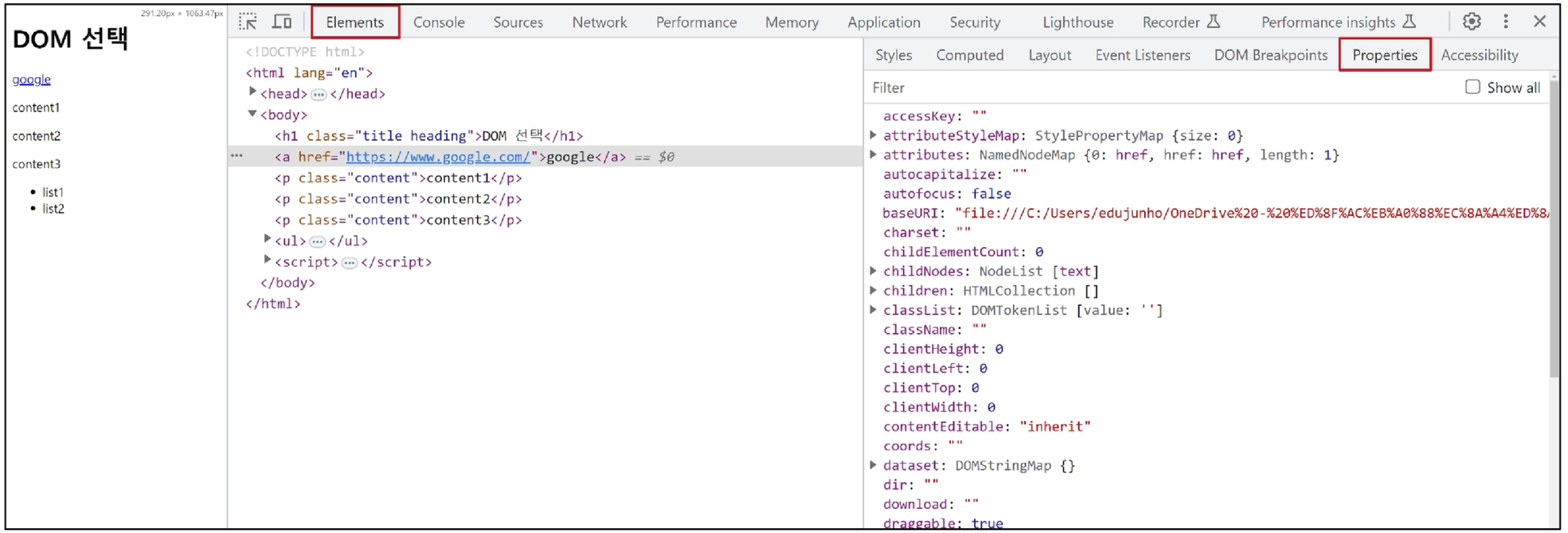
<그림24_Javascript_DOM_style 조작 실습>

참고

DOM 속성 확인 Tip

DOM 속성 확인 Tip

- 개발자도구 - Elements - Properties
- 선택한 해당 요소의 모든 DOM 속성 확인 가능



<그림25_Javascript_DOM_DOM 속성 확인 Tip>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

용어 정리

Node

Node

DOM의 기본 구성 단위

DOM 트리의 각 부분은 Node라는 객체로 표현됨

- Document Node ⟹ HTML 문서 전체를 나타내는 노드
- Element Node ⟹ HTML 요소를 나타내는 노드 (예를 들어 <p>)
- Text Node ⟹ HTML 텍스트 (Element Node 내의 텍스트 컨텐츠를 나타냄)

NodeList

NodeList

DOM 메서드를 사용해 선택한 Node의 목록

- DOM 메서드를 사용해 선택한 Node의 목록
- 배열과 유사한 구조를 가짐
- Index로만 각 항목에 접근 가능
- JavaScript의 배열 메서드 `forEach` 사용 가능
- `querySelectorAll()`에 의해 반환되는 NodeList는 DOM의 변경사항을 실시간으로 반영하지 않음
  - DOM이 나중에 변경되더라도 이전에 이미 선택한 NodeList 값은 변하지 않음

Element

Element

DOM 트리에서 HTML 요소를 나타내는 특별한 유형의 Node

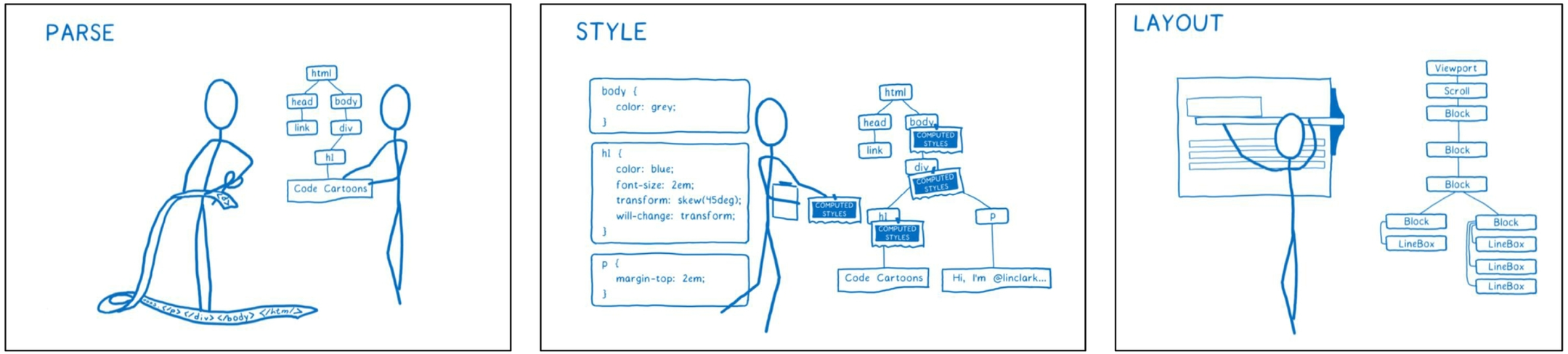
- Node의 하위 유형
- 예를 들어, <p>, , , <body> 등의 HTML 태그들이 Element 노드를 생성
- Node의 속성과 메서드를 모두 가지고 있으며 추가적으로 요소 특화된 기능 (예: className, innerHTML, id 등)을 가지고 있음

➢ 모든 Element는 Node이지만, 모든 Node가 Element인 것은 아님

Parsing

Parsing

브라우저가 문자열을 해석하여
DOM Tree로 만드는 과정
(구문 분석, 해석)



<그림26_Javascript_DOM_Parsing>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

세미콜론

세미콜론 (semicolon)

- 자바스크립트는 문장 마지막 세미콜론(‘`;`’)을 선택적으로 사용 가능
- 세미콜론이 없으면 ASI에 의해 자동으로 세미콜론이 삽입됨
  - ASI (Automatic Semicolon Insertion, 자동 세미콜론 삽입 규칙)
- JavaScript를 만든 Brendan Eich 또한 세미콜론이 선택적이어야 한다고 생각
  (세미콜론을 강제하지 않는 스타일을 선호)

TIP

- 세미콜론의 사용 여부보다 중요한 것은 ‘일관성’ 입니다. 팀의 스타일 가이드를 따르는 것이 우선입니다.

var

var

var

ES6 이전에 변수 선언에 사용했던 키워드

- 재할당 가능
- 재선언 가능
- 함수 스코프(function scope)를 가짐
- “호이스팅” 되는 특성으로 인해 예기치 못한 문제 발생 가능
  - 선언하기 전 사용할 수 있음
- 변수 선언 시 `var`, `const`, `let` 키워드 없이 대입하면 전역 변수처럼 동작하므로 절대 사용하지 않음

함수 스코프

함수 스코프 (function scope)

함수의 중괄호 내부를 가리키고,
함수 스코프를 가지는 변수는 함수 바깥에서 접근 불가

```
function foo() {
      var x = 1
      console.log(x) // 1
}
console.log(x) // ReferenceError: x is not defined
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

호이스팅

호이스팅(hoisting) 이란?

호이스팅

hoisting

코드가 실행되기 전, 변수 및 함수 선언문이
스코프의 최상단으로 끌어올려지는 듯한 현상

모든 선언은 호이스팅 된다.

- `var`, `let`, `const`, `function`, `class` 등 모든 선언은 호이스팅 됨
- 다만, '초기화'가 함께 이루어지느냐에 따라 동작 방식과 에러 발생 여부가 달라짐

변수의 생성 단계와 호이스팅 차이

- 자바스크립트 엔진은 변수를 선언(Declaration) → 초기화(Initialization) → 할당(Assignment) 단계로 생성
- 호이스팅은 이 단계 중 어디까지 진행되느냐가 핵심

1. var의 호이스팅 (선언 + 초기화 동시 진행)
   - **동작:** 스코프 최상단에서 선언과 동시에 undefined로 초기화됨
   - **결과:** 변수 선언 문장 이전에 접근해도 에러가 나지 않고 undefined를 반환

```
console.log(name)  // undefined (에러발생 X)
var name = '홍길동'
```

2. let, const의 호이스팅 (선언만 진행)
   - **동작:** 스코프 최상단으로 선언(이름 등록)은 되지만, 실제 코드 라인에 도달하기 전까지 초기화(메모리 할당)는 되지 않음
   - **결과:** 초기화되지 않은 상태에서 접근하려 하므로 참조 에러(ReferenceError)가 발생

```
console.log(age)  // ReferenceError: Cannot access 'age' before initialization
let age = 30
```

※ **만약 호이스팅이 아예 되지 않았다면 age is not defined (정의되지 않음) 에러가 떠야 하지만,
엔진이 변수의 존재는 알고 있기 때문에 initialization 관련 에러가 발생**

일시적 사각지대 (TDZ, Temporal Dead Zone)

- 스코프의 시작 지점부터 변수의 초기화가 이루어지는 지점(실제 코드 선언부)까지의 구간
- TDZ 참고 링크 (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let#temporal_dead_zone_tdz)

TIP

- `let, const, class`는 호이스팅이 되지만 이 TDZ에 갇혀 있기 때문에, 선언 전에 접근할 수 없습니다.
- 이는 개발자의 실수를 방지하는 안전장치 역할을 합니다.

함수 선언식의 호이스팅

- 함수 선언식(`function a() {}`)은 var보다 더 강력하게 호이스팅 됨
- 선언, 초기화, 할당(함수 본체)이 모두 동시에 최상단으로 끌어올려지므로, 선언 전에도 함수를 문제없이 호출할 수 있음

```
sayHello() // 'Hello!' 출력


function sayHello() {
  console.log('Hello!')
}
```

호이스팅 요약 비교

키워드	호이스팅 여부	초기화 시점	TDZ	비고
var	O	스코프 진입 시 (undefined)	X	선언 전 접근 가능 (권장 X)
let / const	O	실제 코드 선언 줄 도달 시	O	선언 전 접근 불가 (안전)
function	O	스코프 진입 시 (함수 전체)	X	선언 전 호출 가능
함수 표현식	O	실제 코드 선언 줄 도달 시	O	const로 선언하면 let,const 규칙을 따름 (선언 전 호출 불가)
class	O	실제 코드 선언 줄 도달 시	O	선언 전 사용 불가

<표2_Javascript_DOM_호이스팅 요약 비교>

실습

DOM 선택

- 3073. DOM API - HTML 요소 선택과 출력
- 2955. 할일 목록 만들기 - 요소 선택과 출력

DOM 조작

- 3074. DOM API - HTML 속성 및 클래스 속성 조작
- 2953. 포트폴리오 만들기 - DOM 조작
- 2956. 할일 목록 만들기 - 속성 수정과 요소 삭제
- 2958. Card 속성 조작 하기

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 JavaScript의 표준 명세 이름은 무엇인가요?

a) JScript
b) Mocha
c) ECMAScript
d) TypeScript

2 다음 중 변수명으로 사용할 수 없는 것은?

a) _name
b) $user
c) function
d) userName

3 변수 선언 시 기본적으로 사용을 권장하는 키워드는?

a) var
b) for
c) const
d) function

4 웹 페이지(Document)를 객체로 표현한 모델은?

a) HTML
b) CSS
c) API
d) DOM

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 CSS 선택자와 일치하는 첫 번째 요소를 반환하는 메서드는?

a) getElementById()
b) querySelectorAll()
c) getElementsByClassName()
d) querySelector()

6 CSS 선택자와 일치하는 모든 요소를 NodeList로 반환하는 메서드는?

a) querySelector()
b) getElementsByTagName()
c) querySelectorAll()
d) getElementById()

7 요소에 클래스를 추가하는 classList의 메서드는?

a) toggle()
b) remove()
c) add()
d) contains()

8 요소의 속성 값을 설정하는 메서드는?

a) getAttribute()
b) removeAttribute()
c) hasAttribute()
d) setAttribute()

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 요소의 순수한 텍스트 콘텐츠를 조작하는 속성은?

a) innerHTML
b) textContent
c) innerText
d) value

10 새로운 HTML 요소를 메모리에 생성하는 메서드는?

a) appendChild()
b) createElement()
c) removeChild()
d) createTextNode()

11 생성한 요소를 특정 부모 요소의 마지막 자식으로 추가하는 메서드는?

a) insertBefore()
b) replaceChild()
c) appendChild()
d) prepend()

12 var 키워드의 특징이 아닌 것은?

a) 재할당 가능
b) 재선언 가능
c) 블록 스코프
d) 호이스팅

확인 문제

정답 및 해설

1. c) ECMAScript 2. c) function 3. c) const 4. d) DOM 5. d) querySelector() 6. c) querySelectorAll() 7. c) add()
8. d) setAttribute() 9. b) textContent 10. b) createElement() 11. c) appendChild() 12. c) 블록 스코프

1. JavaScript의 파편화를 막기 위해 ECMA 재단에서 만든 표준 언어 명세입니다.
2. function은 함수 선언 키워드로, 프로그래밍 언어가 예약해둔 단어이므로 변수명으로 쓸 수 없습니다.
3. const를 기본으로 사용하여 의도치 않은 재할당을 막고, 필요시에만 let을 사용합니다.
4. DOM(Document Object Model)은 웹 페이지를 구조화된 객체로 제공하는 API입니다.
5. querySelector()는 제공된 선택자를 만족하는 첫 번째 element 객체를 반환합니다.
6. querySelectorAll()은 제공된 선택자를 만족하는 모든 요소를 NodeList로 반환합니다.
7. add() 를 사용하여 지정한 클래스 값을 추가할 수 있습니다.
8. setAttribute()를 사용하여 지정된 요소의 속성 값을 설정하거나 갱신합니다.
9. textContent는 HTML 태그를 제외한 순수한 텍스트 데이터만 가져오거나 설정합니다.
10. createElement()는 요소를 생성하며, appendChild로 문서에 삽입해야 보입니다.
11. appendChild()는 한 노드를 특정 부모 노드의 자식 목록 중 마지막 자식으로 삽입합니다.
12. var는 함수 스코프를 가지며, let과 const가 블록 스코프를 가집니다.

활동 정리

핵심키워드

개념	설명	예시
DOM	HTML 문서를 객체로 표현한 것	document.querySelector()
querySelector	CSS 선택자로 요소 하나를 선택	document.querySelector('.heading')
querySelectorAll	CSS 선택자로 요소 여러 개를 선택	document.querySelectorAll('p')
textContent	요소의 텍스트 내용을 조작	h1.textContent = '수정'
classList	요소의 클래스 목록을 제어	h1.classList.add('red')
setAttribute	요소의 속성 값을 설정	a.setAttribute('href', url)
createElement	새 HTML 요소를 메모리에 생성	document.createElement('h1')

<표1_Javascript_DOM_핵심키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- JavaScript 변수 선언
  - JavaScript에서 변수를 선언할 때는 세 가지 키워드를 사용할 수 있음
    - let: 재할당이 가능한 변수를 선언함(블록 스코프를 가짐)
    - const: 재할당이 불가능한 상수를 선언하며, 선언 시 반드시 초기화해야 함(블록 스코프를 가짐)
    - var: 재선언 및 재할당이 가능하고 함수 스코프를 가지고, 호이스팅 문제로 **현재는 사용을 권장하지 않음**
  - 기본적으로 const를 사용하고, 재할당이 필요한 경우에만 let을 쓰는 것이 좋음

활동 정리

오늘 공부한 내용 요약 및 정리

- DOM (Document Object Model)
  - DOM은 웹 페이지(Document)를 구조화된 객체로 표현한 것임
  - 이를 통해 JavaScript 같은 프로그래밍 언어가 HTML 문서의 구조, 스타일, 내용 등을 동적으로 조작할 수 있음
  - 모든 HTML 요소, 속성, 텍스트는 document 객체 아래의 하위 객체로 구성된 트리 구조(DOM Tree)를 형성
- DOM 요소 선택
  - DOM 조작을 위해서는 먼저 원하는 요소를 선택해야 함
  - document.querySelector(selector): CSS 선택자와 일치하는 첫 번째 요소를 하나 반환함
  - document.querySelectorAll(selector): CSS 선택자와 일치하는 모든 요소를 NodeList 형태로 반환함

활동 정리

오늘 공부한 내용 요약 및 정리

- DOM 요소 조작
  - 선택한 요소의 내용, 속성, 스타일을 변경하거나 새로운 요소를 추가/삭제할 수 있음
  - 콘텐츠 조작: textContent 프로퍼티를 사용해 요소 안의 텍스트를 읽거나 변경함
  - 속성 조작:
    - classList 프로퍼티(add, remove, toggle 메서드)로 클래스를 동적으로 제어함
    - setAttribute(name, value) 메서드로 href 같은 일반 속성의 값을 설정함
  - 요소 조작
    - document.createElement(tagName)로 새 요소를 메모리에 생성함
    - appendChild() 메서드로 생성한 요소를 특정 부모 요소의 마지막 자식으로 추가함
  - 스타일 조작: style 프로퍼티를 사용해 요소의 CSS 스타일을 직접 변경함

활동 정리

웹사이트에서 다크 모드 버튼을 누르면 화면이 어두워지는 건 어떻게 일어나는 걸까요?

- 버튼 클릭 한 번으로 웹 페이지에 변화를 일으키는 방법, 이제 우리는 그 해답을 알고 있습니다.

단 3단계면 우리도 다크 모드를 구현할 수 있습니다.

1. querySelector로 웹 페이지 전체(body)를 선택합니다.
2. classList.toggle()로 'dark-mode' 클래스를 동적으로 추가/제거합니다.
3. 미리 정의된 CSS 스타일에 따라 배경과 글자색이 즉시 변경됩니다.

- 이제 자바스크립트로 웹 페이지의 클래스를 제어하여, 사용자와 상호작용하는 동적인 디자인 변경을 손쉽게 구현할 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다